

MINISTRY OF EDUCATION AND RESEARCH
OF THE REPUBLIC OF MOLDOVA
Technical University of Moldova
Faculty of Computers, Informatics and Microelectronics
Department of Software Engineering and Automation

Sergiu Gherasim, Vladislav Cuibari, FAF-2XX

Report

Laboratory Work No. 1

Network Programming

Topic: Concurrency in Practice

CPU-bound and I/O-bound Work with OS Threads in Java

Checked by:
Cristofor Fistic, *university assistant*

Chişinău – 2026

CONTENTS

Introduction **3**

1 Short Theory **4**
1.1 Threads, Cores and Logical Processors 4
1.2 CPU-bound and I/O-bound Work 4
1.3 Speed-up 4
1.4 Race Conditions and Synchronisation 4
1.5 Producer–Consumer 4

2 Experimental Setup **6**
2.1 Hardware and Software 6
2.2 Data 6
2.3 Build, Run and Measurement 6
2.4 Code Structure 7

3 Task 1a: Counting with Threads **8**
3.1 Implementation 8
3.2 Results 8
3.3 Answers to the Questions 9

4 Task 1b: Three Ways to Count **11**
4.1 Implementation 11
4.2 Results 11
4.3 Answers to the Questions 12

5 Task 1c: Producer and Consumers **13**
5.1 Implementation 13
5.2 Results 13
5.3 Answers to the Questions 14

6 Task 2: Many Files on Slow Storage **16**
6.1 Implementation 16
6.2 Emulated USB Drive 16
6.3 Results 17
6.4 Answers to the Questions 17

7 Control Questions **19**

Conclusions **21**

INTRODUCTION

The purpose of this laboratory work is to measure how the number of threads, and the way the work is divided among them, change the running time and the memory of a real workload. The workload is the same in every task: count the negative, positive and zero values among 50 000 000 integers stored as text. Only the way the work is split, shared and synchronised changes from task to task.

From our own measurements we decide when the work is CPU-bound and when it is I/O-bound, observe a race condition and the cost of fixing it, and find the number of threads at which parallel speed-up stops. All tasks are implemented in one Java program that uses platform (OS) threads; every task and configuration is selected with command-line parameters.

Source code: `https://github.com/Klavrin/network-programming`

1 SHORT THEORY

1.1 Threads, Cores and Logical Processors

A *physical core* is an independent execution unit with its own pipeline, registers and first-level caches. A *logical processor* is what the operating system schedules threads on. With simultaneous multithreading (Hyper-Threading) one physical core exposes two logical processors that share its execution units, so the second one adds far less than a full core. An OS thread can run truly in parallel with another one only when both are placed on different logical processors at the same moment; any additional threads are time-sliced by the scheduler.

1.2 CPU-bound and I/O-bound Work

Work is *CPU-bound* when its running time is determined by the amount of computation, so a faster or a second processor shortens it. It is *I/O-bound* when the program spends most of its time waiting for a device (disk, USB drive, network) to deliver data. For CPU-bound work the useful number of threads is limited by the number of cores. For I/O-bound work concurrency helps only if the waits can overlap, that is, if the device or the remote side can actually serve several requests at the same time.

1.3 Speed-up

If T_1 is the execution time on one thread and T_n the time on n threads, the speed-up is

$$S(n) = \frac{T_1}{T_n}. \tag{1}$$

Ideal linear speed-up is $S(n) = n$. Amdahl's law limits it when a fraction s of the work stays sequential:

$$S(n) \leq \frac{1}{s + (1 - s)/n}. \tag{2}$$

In practice speed-up also stops when the threads exceed the number of cores, or when they compete for a shared resource such as a lock, a memory bus or a storage device.

1.4 Race Conditions and Synchronisation

A race condition occurs when the result depends on the interleaving of threads that access shared data without synchronisation. The statement `counter++` is not atomic: it is a load, an addition and a store. Mutual exclusion (a lock or a `synchronized` block) makes such a sequence atomic, but serialises the threads that use it. Giving every thread its own data and combining the results at the end avoids both problems.

1.5 Producer–Consumer

In the producer–consumer pattern one or more producers put work items into a shared queue and consumers take them out. A *bounded blocking queue* makes the producer wait when the queue is full and the consumers wait when it is empty, which limits memory and balances the load

automatically.

2 EXPERIMENTAL SETUP

2.1 Hardware and Software

CPU	Apple M1 Pro (MacBook Pro)
Physical cores	8: 6 performance (P) cores and 2 efficiency (E) cores
Logical processors	8 (no simultaneous multithreading: one logical processor per core)
RAM	16 GB unified memory
Disk	Internal Apple NVMe SSD (Apple Fabric)
USB drive	None available; emulated as a volume capped at 25 MB/s, USB 2.0 class (Section 6.2)
Operating system	macOS Sonoma 14.8.8 (build 23J620)
Language and runtime	Java, OpenJDK 25 (HotSpot 64-bit Server VM, Homebrew build)
Helper scripts	Python 3.9.6 (data generation, reference totals), Bash

Table 1: Test machine

2.2 Data

The script `number-generator.py` writes `numbers.txt`: 50 000 000 integers drawn uniformly from $[-1\,000\,000;\ 1\,000\,000]$, one per line, 369.4 MB in total. For Task 2, `java -cp java/out Main split` divides the file in order into `part-1.txt` ... `part-8.txt` with exactly 6 250 000 numbers each (about 46.2 MB per file).

The independent single-threaded script `verify.py` produced the reference totals:

`negative = 25 005 851, positive = 24 994 122, zero = 27, total = 50 000 000.`

Every correct run in this report reproduces exactly these numbers. The script `java/test.sh` runs every task and compares its totals with `verify.py` automatically; only Task 1b without synchronisation is expected to differ.

2.3 Build, Run and Measurement

The program needs only a JDK (21 or newer); no build tool is used. All commands are run from the repository root.

```
javac -d java/out $(find java/src -name "*.java")

java -cp java/out Main 1a <threads> [file]                # Task 1a
java -cp java/out Main 1b <unsafe|lock|local> [threads]   # Task 1b
java -cp java/out Main 1c <workers> [file]                # Task 1c
java -cp java/out Main 2  <threads> <directory>           # Task 2
```

```
java/run.sh            # every Task 1 configuration, with peak RSS
java/sim-usb.sh up 25  # Task 2: mount the emulated USB drive
java/sim-usb.sh bench  # Task 2 with 1, 2 and 4 threads
java/test.sh           # correctness check of every task
```

Listing 1: Building and running every task

Time is measured inside the program with `System.nanoTime()` around the counting only; JVM start-up and, in Task 1b, loading the array are excluded. **Peak memory** is reported in two ways. The *process RSS* (maximum resident set size) is measured by `/usr/bin/time -l` and covers everything the process holds: Java heap, thread stacks and the JVM itself. The *peak Java heap* is read from the JVM's memory-pool MXBeans and covers Java objects only. The charts show both.

Every run prints one line, for example:

```
1a threads=8          negative=25005851 positive=24994122 zero=27 total=50000000 time=203ms peakHeap=9MB rss=54MB
```

2.4 Code Structure

```
java/src/
  Main.java                    command line, timing, output
  common/Counts.java           negative / positive / zero counters (not thread-safe on purpose)
  common/NumberReader.java     buffered parser of one integer per line
  common/Parallel.java         run a task on N platform threads, sum their Counts
  common/Memory.java           peak Java heap
  task1a/SplitFileCounter.java each thread parses its own byte range of the file
  task1b/ArrayCounter.java     unsafe / lock / local counting of an in-memory array
  task1c/ProducerConsumer.java reader thread -> ArrayBlockingQueue -> workers
  task2/Splitter.java          makes part-1.txt ... part-8.txt
  task2/ManyFilesCounter.java  each thread reads whole files into one byte array
```

Listing 2: Source layout

All threads are created explicitly with `Thread.ofPlatform()`, so each one is a real OS thread. No thread pool and no virtual threads are used.

3 TASK 1A: COUNTING WITH THREADS

3.1 Implementation

The file is divided by size into as many byte ranges as there are threads: part k of n begins at byte $\lfloor k \cdot size/n \rfloor$. A range boundary usually falls in the middle of a line, so the following rule decides which thread counts each line:

A line belongs to the part that contains its first byte.

Each thread opens the file at one byte before its start and skips everything up to and including the next newline. If the thread starts exactly at the beginning of a line, the byte before it is the previous newline, so only that newline is skipped. Otherwise the thread skips the tail of a line that started in the previous part. The thread then parses lines while the line start is before its end offset, so it also finishes the last line that crosses its end. The first thread starts at offset 0 and skips nothing.

```
long start = Parallel.partStart(size, thread, threads);
long end   = Parallel.partStart(size, thread + 1, threads);
try (NumberReader reader = NumberReader.open(file, Math.max(start - 1, 0))) {
  if (start > 0) reader.skipLine();   // the line containing byte start-1 is not ours
  reader.countUntil(end, counts);     // every line that STARTS before end
}
```

Listing 3: Counting one part (simplified from `SplitFileCounter`)

Every thread reads and parses only its own range, through its own 64 KB buffer, and counts into its own `Counts` object. The results are added after `join()`. The parser works on raw bytes and creates no `String` objects.

3.2 Results

Threads	Time, ms	Speed-up	Peak heap, MB	Peak RSS, MB
1	864	1.00	3	51
2	442	1.95	5	53
4	254	3.40	5	53
8	**203**	**4.26**	9	54
16	218	3.96	13	55
32	316	2.73	25	59
64	320	2.70	45	63

Table 2: Task 1a. All runs: negative = 25 005 851, positive = 24 994 122, zero = 27, total = 50 000 000

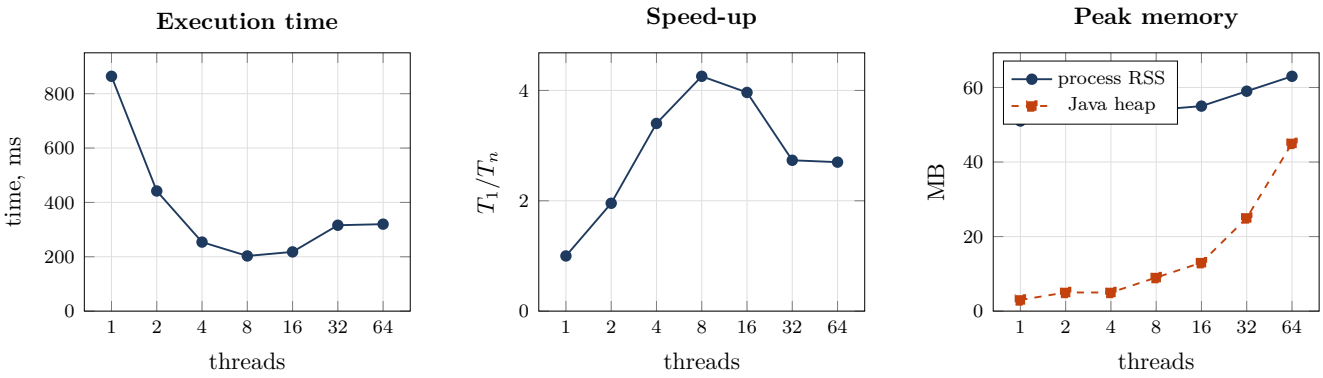


Figure 1: Task 1a: execution time, speed-up and peak memory against the number of threads

3.3 Answers to the Questions

1. At which number of threads does the speed-up stop growing? At **8 threads**, which equals both the number of physical cores and the number of logical processors of the M1 Pro. Up to 4 threads the speed-up is almost linear (1.95 at 2, 3.40 at 4), because each thread gets its own performance core. From 4 to 8 threads it grows much less, from 3.40 to 4.26. The four additional threads include the two efficiency cores, which are considerably slower, and the threads also start to share the memory bandwidth and the last-level cache. A part is finished only when its slowest thread is finished, so the threads on E-cores hold back the result. Beyond 8 threads there is no free processor left and the speed-up falls.

2. Why do 32 and 64 threads not help? Are they slower, and why? Yes, they are slower: 316 and 320 ms against 203 ms at 8 threads, about 55% more. Only 8 threads can run at the same time; the rest wait for the scheduler. The extra threads add no computing power, but they add costs:

- creating, starting and joining 32 or 64 OS threads;
- context switches, after which the incoming thread finds caches filled with another thread's data;
- many small parts: each thread opens its own file channel, allocates its own buffer and reads one partial line twice at its boundary;
- uneven finishing: with more threads than cores, some threads get their CPU time late and the program waits for the last one.

With only 6 MB of text per thread at 64 threads, these fixed costs are no longer small compared with the useful work.

3. Does the peak memory depend on the number of threads? Why? Only slightly. The process RSS grows from 51 to 63 MB and the Java heap from 3 to 45 MB. The program never holds the file in memory: each thread streams its range through a 64 KB buffer. Each extra thread adds its buffer, its file channel and stream objects, and a thread stack (512 KB–1 MB reserved, of which only the touched pages count in RSS). The heap grows faster than the buffers

alone would explain because the run lasts only about 0.2–0.3 s. The garbage collector does not run in that time, so the short-lived objects of 64 threads are all still counted at the peak. The memory therefore grows roughly linearly with the number of threads, but stays small.

4 TASK 1B: THREE WAYS TO COUNT

4.1 Implementation

All 50 000 000 numbers are first loaded into an `int[]` of 200 MB; this step is not measured. The array is divided into 8 equal ranges, one per logical processor, and the three versions differ only in where the counters live:

```
// 1. unsafe: one shared Counts object, no synchronisation
for (int i = from; i < to; i++) shared.add(numbers[i]);

// 2. lock: the same shared object, every update inside a monitor
for (int i = from; i < to; i++) synchronized (shared) { shared.add(numbers[i]); }

// 3. local: every thread has its own Counts; they are added after join()
Counts own = new Counts();
for (int i = from; i < to; i++) own.add(numbers[i]);
return own;
```

Listing 4: The three versions (from `ArrayCounter`)

`Counts.add` performs `negative++`, `positive++` or `zero++` on plain `long` fields; the fields are deliberately neither `volatile` nor atomic.

4.2 Results

Version	Time, ms	Negative	Positive	Zero	Total	Lost	Correct
unsafe, run 1	105	5 080 594	5 071 801	27	10 152 422	79.7%	no
unsafe, run 2	122	4 972 258	4 995 702	27	9 967 987	80.1%	no
unsafe, run 3	82	4 632 441	4 561 836	27	9 194 304	81.6%	no
lock	2525	25 005 851	24 994 122	27	50 000 000	0	yes
local	**76**	25 005 851	24 994 122	27	50 000 000	0	yes

Table 3: Task 1b with 8 threads. Peak heap 453–461 MB and RSS 630 MB in every run, dominated by the 200 MB array

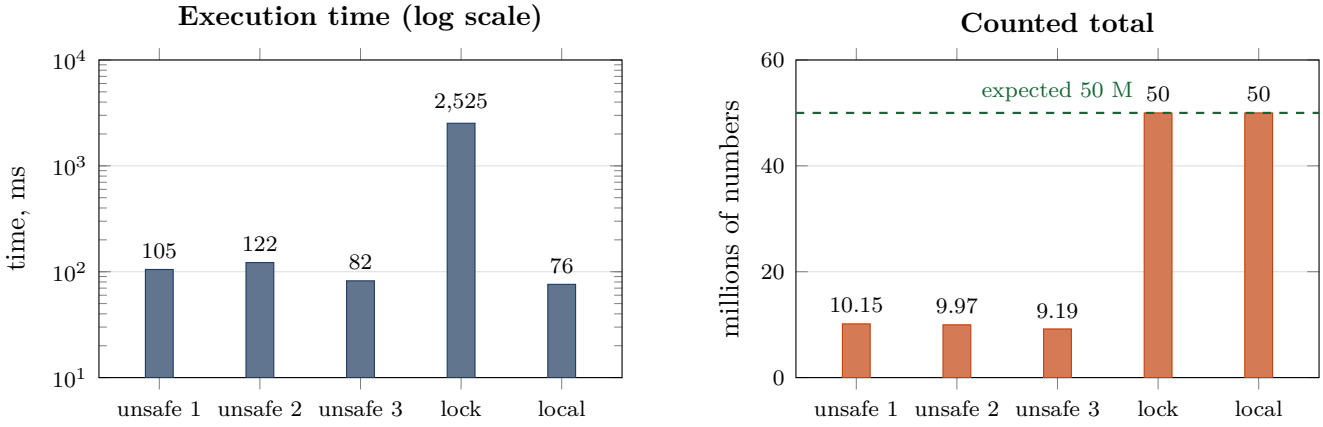


Figure 2: Task 1b: time and counted total of each version (8 threads)

4.3 Answers to the Questions

4. Rank the three versions by time. Which of them are correct? Explain each result.

Ranking from fastest to slowest: **local** (76 ms) < **unsafe** (82–122 ms) < **lock** (2525 ms). Only **local** and **lock** are correct.

- **Local counters: fastest and correct.** Each thread increments counters that no other thread touches. They stay in the thread's registers and its own core's cache. There is no synchronisation and no shared cache line, so the 8 threads really work in parallel. The only coordination is the final addition of 8 results after `join()`, which is correct because `join()` makes all writes of the finished thread visible.

- **Shared counters without synchronisation: incorrect, and not even fast.** About 80% of the increments are lost, and the total changes from run to run (10.15 M, 9.97 M, 9.19 M): this is the race condition. `negative++` is a read–modify–write sequence. When two threads read the same old value, both write back old value +1, and one increment disappears (Section 7, question 3). The loss is so large here because the counters are plain fields: the JIT compiler may keep a counter in a register for many iterations and store it rarely. Each store then overwrites everything the other threads wrote since. `zero = 27` is correct only by chance: with 27 zeros in 50 million numbers two threads practically never increment it at the same moment. The version is also slower than the local one, even though it does less useful work. All 8 cores write to the same object, so its cache line keeps moving from core to core (cache-line contention).

- **Shared counters with a lock: correct but about 33 times slower.** `synchronized` makes each increment atomic and visible, so the totals are exact. But all 50 million increments must acquire the same monitor, one thread at a time. The parallel loop becomes a sequential one plus the cost of acquiring and releasing a contended lock 50 million times, with the lock's cache line bouncing between cores. It is even much slower than one thread without any lock.

5 TASK 1C: PRODUCER AND CONSUMERS

5.1 Implementation

One reader thread reads `numbers.txt` line by line with `BufferedReader.readLine()`, collects 10 000 lines into a `List<String>` and puts the batch into an `ArrayBlockingQueue` with a capacity of 100 batches. `put()` blocks while the queue is full and `take()` blocks while it is empty. At the end of the file the reader puts a special `END` marker (compared by identity), which closes the queue. A worker that takes the marker puts it back for the other workers and stops. Each worker counts into its own `Counts`, which are added after `join()`.

```
// reader
for (String line = in.readLine(); line != null; line = in.readLine()) {
  batch.add(line);
  if (batch.size() == BATCH_SIZE) { queue.put(batch); batch = new ArrayList<>(BATCH_SIZE); }
}
if (!batch.isEmpty()) queue.put(batch);
queue.put(END);                                  // close the queue

// worker
for (List<String> batch = queue.take(); batch != END; batch = queue.take())
  for (String line : batch) counts.add(Integer.parseInt(line.trim()));
queue.put(END);                                  // let the other workers stop too
```

Listing 5: Reader and worker (from `ProducerConsumer`)

5.2 Results

Workers	Time, ms	Speed-up	Peak heap, MB	Peak RSS, MB
1	1890	1.00	152	214
2	1879	1.01	156	218
4	1862	1.02	152	215
8	1887	1.00	152	215
16	1952	0.97	173	226
32	1950	0.97	161	223
64	1908	0.99	153	222

Table 4: Task 1c. All runs: negative = 25 005 851, positive = 24 994 122, zero = 27, total = 50 000 000

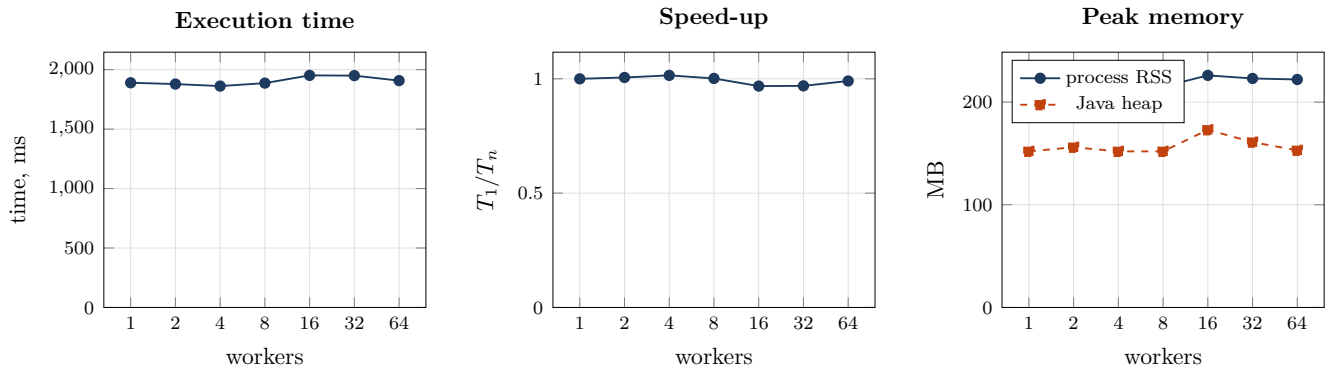


Figure 3: Task 1c: execution time, speed-up and peak memory against the number of workers

5.3 Answers to the Questions

3. Does the peak memory depend on the number of threads? Why? *(for Task 1c)* No. The RSS stays between 214 and 226 MB for 1 to 64 workers. Memory is determined by the queue, not by the workers. At most 100 batches of 10 000 lines wait in the queue, plus the batch being filled and one batch per worker. That is about one million `String` objects of roughly 50 bytes each, plus the `ArrayLists` and the garbage of batches already counted. The bounded queue is exactly what keeps this number constant.

5. Compare Task 1c with Task 1a. Which one is faster, and why? At which number of workers does Task 1c stop getting faster? Task 1a is much faster: 203 ms at its best against about 1870 ms, a factor of 9. Task 1c is even slower than Task 1a on a single thread (864 ms). In Task 1c the reading and the splitting into lines are done by one thread, which must decode the bytes into characters and create a new `String` for each of the 50 million lines. That sequential part takes almost the whole 1.9 s. Parsing a short string with `Integer.parseInt` is cheap, so a single worker already keeps up with the reader and the other workers mostly wait on an empty queue. Task 1c therefore stops getting faster at **1 worker**: the time is flat from 1 to 8 workers, and at 16–64 workers it grows slightly because of thread creation and contention on the queue's lock. By Amdahl's law the sequential reader bounds the speed-up to about 1. Task 1a, in contrast, splits the reading itself, parses raw bytes without creating objects and has no shared data structure at all.

6. What would happen with batches of a single line? And with a queue that has no size limit?

- **Batches of one line.** Every number would cost one `put()` and one `take()`: 50 million lock acquisitions on each side of the queue, frequent wake-ups of blocked threads and 50 million extra list objects. The synchronisation overhead would be far larger than the work of counting one number. The program would become several times slower, and slower still with more workers, because they would all contend for the same queue lock. Batching amortises the cost of the queue over 10 000 numbers.
- **A queue without a size limit.** The reader would never block. Whenever it is faster than the workers, it would run ahead and accumulate the unprocessed part of the file in memory.

In the worst case that is all 50 million lines as `String` objects, several gigabytes of heap. The time would barely improve and the program could fail with `OutOfMemoryError`. The bound on the queue is the back-pressure that keeps memory constant.

6 TASK 2: MANY FILES ON SLOW STORAGE

6.1 Implementation

A file is the unit of work. Thread t of n processes files $t, t+n, t+2n, \ldots$, so the 8 files are divided as evenly as possible: 8 per thread with 1 thread, 4 with 2 and 2 with 4. Each file is read in a single operation into one byte array with `Files.readAllBytes`, then parsed from memory and counted into the thread's own counters.

```
for (int i = thread; i < files.size(); i += threads) {
  byte[] bytes = Files.readAllBytes(files.get(i));       // one read, one buffer
  try (NumberReader reader = new NumberReader(new ByteArrayInputStream(bytes))) {
    reader.countUntil(Long.MAX_VALUE, counts);
  }
}
```

Listing 6: Task 2 worker (from `ManyFilesCounter`)

6.2 Emulated USB Drive

The handout requires the 8 files to be read from a USB flash drive. We did not have one, so we emulated a USB 2.0 drive. The goal was to keep the properties that matter for the experiment:

1. the files are read by the unchanged program from a **real mounted volume** through the kernel's file system;
2. the volume has **one fixed total bandwidth**, shared by all threads like a single USB bus;
3. **no data comes from the file cache**: the drive can be "unplugged" before every run;
4. Spotlight does not index the drive.

The script `java/sim-usb.sh` packs the 8 files into a read-only, uncompressed HFS+ disk image. A small HTTP server, `java/usb-server.py`, serves the image on `127.0.0.1` and sends at most 25 MB/s in total, whatever the number of connections: all transfers take turns on one shared time budget. macOS mounts the image directly from that URL with `hdiutil attach`, so every read that misses the cache goes through the rate-limited server. Before every run the script detaches and re-attaches the image, which empties the cache of that volume. This is equivalent to ejecting, unplugging and re-inserting the drive. The volume is mounted with `-nobrowse`, is read-only and contains `.metadata_never_index`, so Spotlight does not touch it.

```
java/sim-usb.sh up 25    # build image, start server, mount at .sim-usb/mnt
java/sim-usb.sh bench    # re-plug, run 1 thread; re-plug, 2 threads; re-plug, 4 threads
java/sim-usb.sh down     # unmount and stop the server
```

Listing 7: Running Task 2 on the emulated drive

We checked that the emulation behaves like a removable drive. A cold read of one 46 MB file took 2.35 s (19.6 MB/s), while an immediate second read of the same file took 0.003 s from

the cache. This shows that re-plugging before each run is necessary, and that it works. The effective throughput of about 19 MB/s is below the configured 25 MB/s because of the HTTP and disk-image overhead, which is typical for a real USB 2.0 flash drive.

6.3 Results

Threads	Time, ms	Speed-up	Throughput, MB/s	Peak heap, MB	Peak RSS, MB
1	20 027	1.00	18.4	97	185
2	19 124	1.05	19.3	97	230
4	19 195	1.04	19.2	327	544
Reference, Task 1a on the internal SSD, 1 thread: 864 ms, 427.6 MB/s					

Table 5: Task 2 on the emulated USB drive, re-plugged before every run. All runs: negative = 25 005 851, positive = 24 994 122, zero = 27, total = 50 000 000

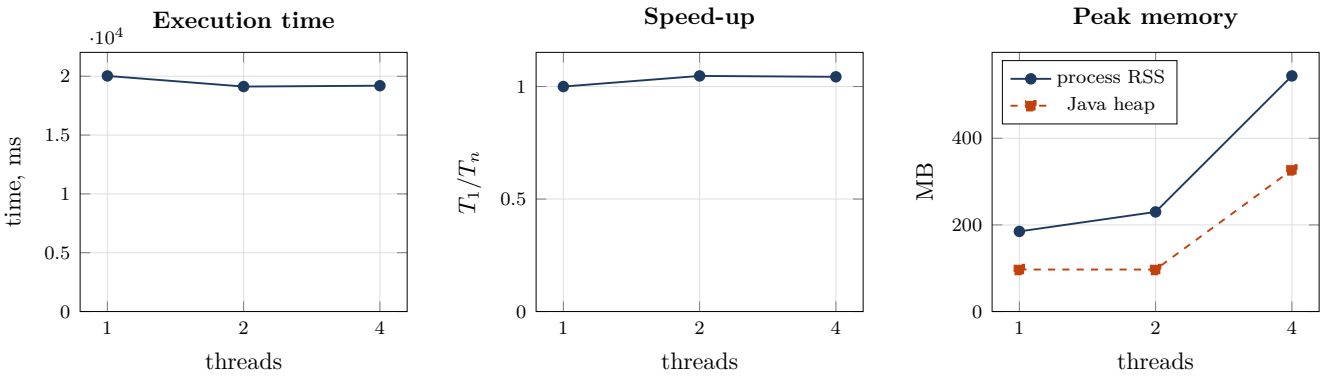


Figure 4: Task 2: execution time, speed-up and peak memory against the number of threads

6.4 Answers to the Questions

1. Compare the time on one thread with Task 1a on one thread. On the USB drive, is the work CPU-bound or I/O-bound? It is **I/O-bound**. Both runs parse exactly the same 50 million numbers with the same parser on one thread. From the internal SSD this takes 864 ms; from the USB drive it takes 20 027 ms, 23 times longer. The computation did not change, so the extra 19.2 s are spent waiting for data. Parsing accounts for at most 0.864/20.03 ≈ 4% of the run and waiting for the drive for about 96%. The measured rate, 369.4 MB/20.03 s ≈ 18.4 MB/s, equals the drive's own speed. On the SSD the same code processes 428 MB/s and is limited by the processor. The work is therefore CPU-bound on the SSD and I/O-bound on the USB drive.

2. On the USB drive, do more threads help? Why not, when lecture 1 showed that I/O-bound work benefits from concurrency? Practically not: 2 threads give a speed-up of 1.05 and 4 threads 1.04, while the memory grows from 185 to 544 MB. The small gain from 1 to 2 threads is exactly the parsing time. With one thread, reading and parsing alternate

(19.1 s + 0.9 s ≈ 20.0 s). With two threads, one thread parses while the other reads, so the 0.9 s of parsing is hidden behind the transfer. After that, only the transfer remains, and its speed is fixed: all threads read from the same device over the same bus. With 4 threads each one simply receives a quarter of the same 19 MB/s.

Concurrency helps I/O-bound work only when the thing being waited for can serve several requests at the same time. That is the case for a server waiting on many independent remote clients, where the waits overlap and the total time shrinks. A USB flash drive is a single device with a single, already saturated channel: a second request does not make the bus faster, it only splits it. Extra threads even risk making the drive slower by turning sequential reads into interleaved reads at different positions. Memory is the price paid for nothing: every thread holds a whole 46 MB file in its byte array, so at 4 threads four files, plus garbage not yet collected, are in memory at once.

7 CONTROL QUESTIONS

1. What is the difference between a physical core and a logical processor? Where did it show in your charts? A physical core is a complete execution unit with its own pipeline, registers and L1 caches. A logical processor is what the operating system sees and schedules threads on. With simultaneous multithreading one core appears as two logical processors that share its execution units, so the second one typically adds only 10–30%. Our M1 Pro has no SMT: it has 8 cores and 8 logical processors. Its cores are not identical, though: 6 are fast performance cores and 2 are slower efficiency cores. In Figure 1 this shows as near-linear speed-up up to 4 threads, a much flatter segment from 4 to 8 threads where the efficiency cores join, and the peak at exactly 8 threads, the number of logical processors. After it, threads only share the same 8 processors and the speed-up falls.

2. How did you make sure that no line was counted twice or skipped at the boundary between two parts of the file? With one rule: a line belongs to the part that contains its first byte. Every thread starts one byte before its range and skips up to and including the next newline. If its range starts at a line beginning, it skips only the previous newline. If it starts in the middle of a line, it skips the rest of that line, which the previous thread counts. Every thread then counts each line that *starts* before its end offset, including the one that runs past the end. Each line therefore has exactly one owner. Two independent checks confirm it: all 7 thread counts in Task 1a give exactly 50 000 000 numbers with the same split as `verify.py`, and `java/test.sh` compares every run automatically.

3. Explain step by step how unsynchronised shared counters lose updates. `positive++` is three operations: load the value from memory into a register, add 1, store it back. Suppose `positive = 100`:

1. thread A loads 100;
2. thread B loads 100, because A has not stored yet;
3. A adds 1 and stores 101;
4. B adds 1 and stores 101.

Two numbers were counted, but the counter grew by one: B's store overwrote A's update. Without synchronisation there is also no visibility guarantee, so the JIT may keep the counter in a register for many iterations and write it back rarely. Then a single store can erase thousands of other threads' increments. This is why our unsafe version lost about 80% of the counts, with a different total on every run.

4. Why are per-thread counters faster than shared counters protected by a lock? With a lock, every one of the 50 million increments must acquire and release the same monitor. Only one thread can hold it, so the threads run one after another, and each acquisition is an atomic operation on a cache line that keeps moving between cores. A contended lock may even

park threads in the operating system. Per-thread counters need no synchronisation during the loop: each thread works only in its own registers and cache. The results are combined once, eight additions after `join()`. In Task 1b this was 76 ms against 2525 ms.

5. What does "I/O-bound" mean for a USB drive, and for a server waiting on the network? Why does concurrency help in only one of the two cases? In both cases the CPU is mostly idle and the time is spent waiting. The difference is what the program waits for. A USB drive is a single, local resource with a fixed bandwidth that one sequential reader already saturates. More threads only divide that bandwidth, so the total time stays the same, as in Task 2. A server waiting on the network waits for many independent remote parties: other servers, databases, clients, each with its own latency. These waits do not compete for one bottleneck, so they can overlap. While one request waits, another proceeds, and with n concurrent requests the total time approaches that of the slowest one instead of the sum. Concurrency helps when the waiting is for latency on independent resources, and does not help when the waiting is for the throughput of one saturated resource.

6. What is the difference between dividing the work into equal parts in advance and taking it from a queue? When is each one better? Static division (Task 1a, Task 2) gives each thread a fixed share at the start. It needs no coordination while running and has no shared data structure. Its weakness is load imbalance: if one part is slower or one thread runs on a slower core, everyone waits for it. It is best when the work is uniform and can be split cheaply, as with a file split by byte offsets. Dynamic division through a queue (Task 1c) lets each thread take the next piece when it is free. Faster threads automatically do more, and the producer can start before the total amount of work is known, for example when reading a stream. It costs synchronisation on the queue for every item, which is why batches must be large enough, and the producer itself can become the bottleneck. It is best when the pieces have unpredictable cost, the threads run at different speeds, or the work arrives as a stream.

CONCLUSIONS

All tasks produced the same totals, 25 005 851 negative, 24 994 122 positive and 27 zero values, adding up to 50 000 000 and identical to the independent reference script. The only exception was the deliberately unsynchronised version of Task 1b, which lost about 80% of the updates and gave a different total on each run.

On our machine, **counting from the internal SSD was CPU-bound**. Task 1a scaled almost linearly to 4 threads, reached its maximum speed-up of 4.26 at 8 threads, exactly the number of cores and logical processors, and became slower with 32 and 64 threads. We know it is CPU-bound because adding processors shortened the time while the amount of data read stayed the same. The SSD delivered data at more than 400 MB/s, so it was never the limit. The heterogeneous M1 Pro, with 6 performance and 2 efficiency cores, showed in the curve as a weaker gain between 4 and 8 threads.

Reading from the (emulated) USB drive was I/O-bound. The same parsing on one thread took 20.0 s instead of 0.86 s. The throughput of 18–19 MB/s matched the drive's speed, and 2 or 4 threads improved the time by only 4–5%, which is just the overlap of parsing with the transfer. We know it is I/O-bound because the computation was unchanged while the time grew 23 times, and because more threads did not increase the throughput of the single shared device. They only increased the memory, from 185 to 544 MB.

The experiments also showed that **the way the work is shared matters as much as the number of threads**. Shared mutable counters are either wrong (no lock) or 33 times slower (one lock), while per-thread counters are both correct and fastest. A producer–consumer pipeline (Task 1c) is limited by its single producer: with one reader creating 50 million strings, adding workers beyond the first changed nothing, and it was 9 times slower than splitting the file itself among the threads. A bounded queue keeps its memory constant regardless of the number of workers.